

RAG Evaluation Harness — Production-Grade Implementation Guide

Section 1: Project Architecture & Overview

1.1 System Design Narrative

The RAG Evaluation Harness is a rigorous, end-to-end pipeline that scores Retrieval-Augmented Generation outputs across four standard quality metrics using an LLM-as-judge pattern. The system flows through the following topology:

1. **Raw Corpus Ingestion:** Plain-text `.txt` files are dropped into `data/raw/` by the intern.
2. **Chunking & Embedding:** Each file is split into overlapping word-level chunks. Every chunk is embedded using `sentence-transformers/all-MiniLM-L6-v2` and stored in a persistent ChromaDB collection named `rag_corpus`.
3. **Question Ingestion:** The evaluation test set (`eval/test_set.jsonl`) contains 30 questions, each paired with a ground-truth answer and the IDs of the chunks that ideally should have been retrieved.
4. **RAG Retrieval + Generation:** For each question, the query is embedded, ChromaDB performs a cosine-similarity search returning `top_k` chunks, and `llama-3.3-70b-versatile` (via Groq API) generates a grounded answer using only the retrieved context.
5. **Parallel Judge Evaluation:** Four independent LLM-as-judge calls score the output on Faithfulness, Answer Relevance, Context Precision, and Context Recall. Each returns a float in `[0.0, 1.0]` plus reasoning text.
6. **CSV Aggregation:** All scores are collected into `eval/results.csv` with 4 decimal precision.
7. **Markdown Report Generation:** `eval/eval_report.md` is programmatically generated, containing aggregate statistics, a score-distribution histogram, best/worst case deep-dives, and a prioritized improvement roadmap.

1.2 Two-Pipeline Distinction

Indexing Pipeline (`src/ingest.py`) — Run once, offline:

- Loads raw corpus, chunks text, generates deterministic chunk IDs, embeds chunks, and upserts them into ChromaDB.
- This is expensive and stateful. In production, it runs on a CI/CD trigger or scheduled batch job when documents change.

Evaluation Pipeline (`eval/run_eval.py`) — Run per eval cycle:

- Loads the pre-built ChromaDB index, ingests the test set, runs RAG + judging, and produces the CSV and markdown report.
- This separation matters because re-embedding the entire corpus for every evaluation is wasteful and non-deterministic. The index is a stable artifact; the evaluation is a reproducible measurement against that artifact.

1.3 LLM-as-Judge Pattern

Using an LLM as a judge is the industry standard for RAG evaluation because rule-based metrics (BLEU, ROUGE, F1) measure lexical overlap, not semantic correctness. A RAG answer can be fully correct while using completely different words from the ground truth, or it can be fluent and wrong. LLM judges evaluate meaning, coverage, and factual grounding.

Advantages over rule-based metrics:

- Captures semantic nuance, paraphrasing, and inference.
- Can assess faithfulness against retrieved context (no reference answer needed).
- Produces human-readable reasoning chains.

Key limitation — Judge bias / self-evaluation risk:

- If the same model family generates the answer and judges it, the judge may favor its own style or fail to catch subtle hallucinations it would have produced itself. This is known as *inbreeding bias*. Mitigation strategies include prompt hardening, temperature=0, and periodic human calibration audits.

1.4 Library Rationale Table

Library	Version	Why This Specific Choice	Alternatives Considered
<code>groq</code>	<code>>=0.9.0</code>	Official SDK for Groq API; handles streaming, retries, and auth cleanly. Required for both generation and judging.	OpenAI SDK (different API shape), raw HTTP requests (too brittle)
<code>chromadb</code>	<code>>=0.5.0</code>	Lightweight, persistent local vector store with Python-native API. Excellent for intern projects and prototyping.	Pinecone (cloud-only, API key complexity), Weaviate (Docker overhead), FAISS (no persistence layer)
<code>sentence-transformers</code>	<code>>=3.0.0</code>	Industry standard for local embedding inference. <code>all-MiniLM-L6-v2</code> is fast, small, and high-quality for general domain.	OpenAI text-embedding-3 (requires API calls, cost), Instructor-XL (slower, larger)
<code>pandas</code>	<code>>=2.0.0</code>	De-facto standard for tabular data manipulation and CSV export.	Polars (faster but less familiar), raw <code>csv</code> module (too primitive)
<code>matplotlib</code>	<code>>=3.8.0</code>	Stable, well-documented plotting for histogram generation.	Seaborn (higher-level but unnecessary for simple histograms), Plotly (overkill for static reports)
<code>jsonlines</code>	<code>>=4.0.0</code>	Clean streaming reader/writer for JSON Lines format without manual line splitting.	Manual <code>json.loads</code> per line (error-prone, no streaming abstraction)

Library	Version	Why This Specific Choice	Alternatives Considered
python-dotenv	>=1.0.0	Loads <code>.env</code> files into environment variables transparently. Zero-config secret management.	Manual <code>os.environ</code> parsing, <code>direnv</code> (shell dependency)
tqdm	>=4.66.0	Lightweight progress bars for long-running loops (embedding, evaluation).	Rich (heavier dependency, overkill), manual print (poor UX)

1.5 RAGAS Reference

RAGAS (Retrieval-Augmented Generation Assessment) is an open-source framework (docs.ragas.io) that automates RAG evaluation using LLM-as-judge for metrics including faithfulness, answer relevance, context precision, and context recall. This project manually reimplements RAGAS's core concepts from scratch.

Why build from scratch instead of using RAGAS directly:

- Building the harness manually forces deep understanding of every pipeline stage: chunking strategy, embedding mechanics, prompt engineering, JSON parsing fragility, and error propagation.
- It teaches how to design resilient production systems (graceful degradation, centralized logging, deterministic IDs) that black-box frameworks hide.
- It provides a customizable foundation: intern-defined rubrics, custom report formats, and tight integration with the Groq API constraint.

Section 2: Repository & Folder Structure

2.1 ASCII Tree Diagram

```
rag-eval-harness/  
├── myenv/  
├── data/  
│   └── raw/                # Intern drops .txt corpus files here  
├── src/  
│   ├── __init__.py  
│   ├── ingest.py           # Chunking + ChromaDB indexing pipeline  
│   ├── rag_bot.py          # RAG retrieval + generation logic  
│   └── utils.py            # Shared utilities (logging setup, token  
counting)  
├── eval/  
│   ├── test_set.jsonl      # 30 questions with ground-truth answers +  
chunk_ids  
│   ├── judges.py           # 4 judge functions (one per metric)  
│   ├── run_eval.py         # Master evaluation orchestrator  
│   ├── results.csv         # Auto-generated: scores for all 30 questions  
│   └── eval_report.md      # Auto-generated: aggregate analysis +  
best/worst cases
```

```

├── tests/
│   ├── test_judges.py
│   └── test_rag_bot.py
├── config/
│   └── settings.py
top_k, thresholds
├── .env
├── .gitignore
├── requirements.txt
├── output.log
└── guide.md
# Central config: model names, chunk size,
# GROQ_API_KEY lives here (never commit this)
# Auto-created: all logs mirrored here

```

2.2 One-Shot Bash Setup Script

```

# setup.sh
#!/bin/bash
set -e

PROJECT_NAME="rag-eval-harness"
echo "Setting up $PROJECT_NAME..."

# Create directory structure
mkdir -p data/raw
mkdir -p src
mkdir -p eval
mkdir -p tests
mkdir -p config

# Create empty Python package and source files
touch src/__init__.py
touch src/ingest.py
touch src/rag_bot.py
touch src/utils.py
touch eval/judges.py
touch eval/run_eval.py
touch eval/test_set.jsonl
touch eval/results.csv
touch eval/eval_report.md
touch tests/test_judges.py
touch tests/test_rag_bot.py
touch config/settings.py
touch requirements.txt
touch guide.md
touch output.log

# Create virtual environment (name: myenv, NOT venv)
python3 -m venv myenv

# Create .gitignore
cat > .gitignore << 'EOF'
myenv/

```

```
.env
output.log
__pycache__/
*.pyc
chroma_db/
EOF

# Create .env.example
cat > .env.example << 'EOF'
GROQ_API_KEY=your_key_here
EOF

# Create requirements.txt
cat > requirements.txt << 'EOF'
groq>=0.9.0
chromadb>=0.5.0
sentence-transformers>=3.0.0
pandas>=2.0.0
matplotlib>=3.8.0
jsonlines>=4.0.0
python-dotenv>=1.0.0
tqdm>=4.66.0
pytest>=8.0.0
EOF

# Install dependencies inside the virtual environment
source myenv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

echo "Setup complete. Activate the environment with: source
myenv/bin/activate"
```

Section 3: Production-Ready Implementation Code

A. `config/settings.py`

```
# config/settings.py
from typing import Final

GROQ_MODEL: Final[str] = "llama-3.3-70b-versatile"
EMBEDDING_MODEL: Final[str] = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE: Final[int] = 512
CHUNK_OVERLAP: Final[int] = 128
TOP_K_RETRIEVAL: Final[int] = 5
CHROMA_PERSIST_DIR: Final[str] = "chroma_db"
EVAL_TEST_SET_PATH: Final[str] = "eval/test_set.jsonl"
EVAL_RESULTS_PATH: Final[str] = "eval/results.csv"
```

```
LOG_FILE_PATH: Final[str] = "output.log"
SCORE_THRESHOLD: Final[float] = 0.6
```

B. src/utils.py

```
# src/utils.py
import json
import logging
import sys
from pathlib import Path

from config.settings import LOG_FILE_PATH

def setup_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)

    if logger.hasHandlers():
        logger.handlers.clear()

    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

    stream_handler = logging.StreamHandler(sys.stdout)
    stream_handler.setLevel(logging.INFO)
    stream_handler.setFormatter(formatter)
    logger.addHandler(stream_handler)

    file_handler = logging.FileHandler(LOG_FILE_PATH, mode="a")
    file_handler.setLevel(logging.INFO)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

    return logger

def count_tokens(text: str) -> int:
    # Heuristic: 1 word ≈ 1.3 tokens on average for English text.
    # This is a fast approximation. For production, use tiktoken or the
    # model's tokenizer.
    return int(len(text.split()) * 1.3)

def load_jsonl(path: str) -> list[dict]:
    logger = setup_logger(__name__)
    data = []
    try:
        with open(path, "r", encoding="utf-8") as f:
```

```

        for line_num, line in enumerate(f, 1):
            line = line.strip()
            if not line:
                continue
            try:
                data.append(json.loads(line))
            except json.JSONDecodeError as e:
                logger.error(f"Invalid JSON on line {line_num} in
{path}: {e}")
        except FileNotFoundError:
            logger.error(f"File not found: {path}")
            raise
    except Exception as e:
        logger.error(f"Error reading {path}: {e}")
        raise
    return data

def save_jsonl(data: list[dict], path: str) -> None:
    logger = setup_logger(__name__)
    try:
        Path(path).parent.mkdir(parents=True, exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            for item in data:
                f.write(json.dumps(item, ensure_ascii=False) + "\n")
            logger.info(f"Saved {len(data)} records to {path}")
    except Exception as e:
        logger.error(f"Error writing to {path}: {e}")
        raise

```

C. src/ingest.py

```

# src/ingest.py
import hashlib
import time
from pathlib import Path

import chromadb
from sentence_transformers import SentenceTransformer
from tqdm import tqdm

from config import settings
from src.utils import setup_logger, count_tokens

logger = setup_logger(__name__)

def load_corpus(data_dir: str) -> list[str]:
    texts = []
    data_path = Path(data_dir)
    if not data_path.exists():

```

```

        logger.error(f"Data directory not found: {data_dir}")
        raise FileNotFoundError(f"Data directory not found: {data_dir}")

txt_files = sorted(data_path.glob("*.txt"))
total_chars = 0

for txt_file in txt_files:
    try:
        with open(txt_file, "r", encoding="utf-8") as f:
            text = f.read()
            texts.append(text)
            total_chars += len(text)
            logger.info(f"Loaded {txt_file.name}: {len(text)} chars")
    except Exception as e:
        logger.error(f"Failed to read {txt_file.name}: {e}")

logger.info(f"Loaded {len(texts)} files, {total_chars} total
characters")
return texts

def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
    words = text.split()
    chunks = []
    if not words:
        return chunks

    start = 0
    while start < len(words):
        end = min(start + chunk_size, len(words))
        chunk_words = words[start:end]
        chunks.append(" ".join(chunk_words))
        if end == len(words):
            break
        start += chunk_size - overlap

    return chunks

def generate_chunk_id(text: str, index: int) -> str:
    content = f"{index}:{text[:50]}"
    return hashlib.md5(content.encode("utf-8")).hexdigest()

def build_chroma_index(chunks: list[str], chunk_ids: list[str]) ->
chromadb.Collection:
    client = chromadb.PersistentClient(path=settings.CHROMA_PERSIST_DIR)
    collection = client.get_or_create_collection(name="rag_corpus")

    embedder = SentenceTransformer(settings.EMBEDDING_MODEL)

    batch_size = 100
    total_batches = (len(chunks) + batch_size - 1) // batch_size
    for i in tqdm(range(0, len(chunks), batch_size), desc="Embedding &

```

```

indexing"):
    batch_chunks = chunks[i : i + batch_size]
    batch_ids = chunk_ids[i : i + batch_size]
    embeddings = embedder.encode(batch_chunks,
show_progress_bar=False).tolist()
    collection.upsert(ids=batch_ids, embeddings=embeddings,
documents=batch_chunks)
    logger.info(f"Indexed batch {i // batch_size + 1}/{total_batches}")

    logger.info(f"Indexed {len(chunks)} chunks into ChromaDB collection
'rag_corpus'")
    return collection

def main():
    logger.info("=== Ingestion Pipeline Started ===")
    start = time.time()

    raw_texts = load_corpus("data/raw")
    all_chunks = []
    all_chunk_ids = []

    for doc_idx, text in enumerate(raw_texts):
        doc_chunks = chunk_text(text, settings.CHUNK_SIZE,
settings.CHUNK_OVERLAP)
        base_idx = len(all_chunks)
        for chunk_idx, chunk in enumerate(doc_chunks):
            global_idx = base_idx + chunk_idx
            chunk_id = generate_chunk_id(chunk, global_idx)
            all_chunks.append(chunk)
            all_chunk_ids.append(chunk_id)
        logger.info(f"Document {doc_idx}: chunked into {len(doc_chunks)}
chunks")

    collection = build_chroma_index(all_chunks, all_chunk_ids)
    elapsed = time.time() - start
    logger.info(f"=== Ingestion Pipeline Completed in {elapsed:.2f}s ===")

if __name__ == "__main__":
    main()

```

D. src/rag_bot.py

```

# src/rag_bot.py
import chromadb
from groq import Groq
from sentence_transformers import SentenceTransformer

from config import settings
from src.utils import setup_logger

```

```

logger = setup_logger(__name__)

_embedder = None

def _get_embedder():
    global _embedder
    if _embedder is None:
        _embedder = SentenceTransformer(settings.EMBEDDING_MODEL)
    return _embedder

def retrieve(
    query: str, collection: chromadb.Collection, top_k: int
) -> tuple[list[str], list[str]]:
    embedder = _get_embedder()
    query_embedding = embedder.encode([query],
    show_progress_bar=False).tolist()[0]

    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k,
        include=["documents"],
    )

    documents = results["documents"][0] if results["documents"] else []
    ids = results["ids"][0] if results["ids"] else []

    logger.info(f"Retrieved {len(documents)} chunks for query")
    return documents, ids

def generate_answer(question: str, context_chunks: list[str], client: Groq)
-> str:
    context_text = "\n\n---\n\n".join(context_chunks)

    system_prompt = (
        "You are a grounded assistant. Answer ONLY using the provided\n\ncontext. "\n\n
        "If the context does not contain the answer, say: 'I cannot answer\n\nthis from the provided context.' "\n\n
        "Do not use any outside knowledge."
    )

    user_prompt = f"Context:\n{context_text}\n\nQuestion:\n{question}\n\nAnswer:"

    try:
        response = client.chat.completions.create(
            model=settings.GROQ_MODEL,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ]
        )

```

```

        ],
        temperature=0.0,
        max_tokens=1024,
    )
    answer = response.choices[0].message.content.strip()
    logger.info(f"Generated answer ({len(answer)} chars)")
    return answer
except Exception as e:
    logger.error(f"Groq API error during generation: {e}",
exc_info=True)
    return "ERROR: Generation failed."

def get_rag_response(
    question: str, collection: chromadb.Collection, client: Groq
) -> dict:
    try:
        retrieved_chunks, retrieved_chunk_ids = retrieve(
            question, collection, settings.TOP_K_RETRIEVAL
        )
        answer = generate_answer(question, retrieved_chunks, client)
        return {
            "question": question,
            "answer": answer,
            "retrieved_chunks": retrieved_chunks,
            "retrieved_chunk_ids": retrieved_chunk_ids,
        }
    except Exception as e:
        logger.error(
            f"RAG pipeline failed for question '{question[:50]}...': {e}",
            exc_info=True,
        )
        return {
            "question": question,
            "answer": "ERROR: RAG pipeline failed.",
            "retrieved_chunks": [],
            "retrieved_chunk_ids": [],
        }

```

E. eval/judges.py

```

# eval/judges.py
import json
import re

from groq import Groq

from config import settings
from src.utils import setup_logger

logger = setup_logger(__name__)

```

```

def _parse_judge_response(response_text: str) -> dict:
    """
    Defensive JSON parser for LLM judge outputs.
    Strips markdown fences, preamble, and extracts the first JSON object.
    """
    text = response_text.strip()

    if text.startswith("`json`"):
        text = text[7:]
    elif text.startswith("`"):
        text = text[3:]
    if text.endswith("`"):
        text = text[:-3]
    text = text.strip()

    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    match = re.search(r"\{.*\}", text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass

    raise ValueError(f"Could not extract valid JSON from response:
{response_text[:200]}")

def _call_judge(prompt: str, client: Groq) -> dict:
    try:
        response = client.chat.completions.create(
            model=settings.GROQ_MODEL,
            messages=[
                {
                    "role": "system",
                    "content": "You are an expert evaluator. Output ONLY
valid JSON.",
                },
                {"role": "user", "content": prompt},
            ],
            temperature=0.0,
            max_tokens=512,
        )
        raw = response.choices[0].message.content.strip()
        parsed = _parse_judge_response(raw)

        score = parsed.get("score")
        reasoning = parsed.get("reasoning", "No reasoning provided.")
    
```

```

        if score is None:
            raise ValueError("Missing 'score' key in judge response")

        try:
            score = float(score)
        except (ValueError, TypeError):
            raise ValueError(f"Score is not a float: {score}")

        if not (0.0 <= score <= 1.0):
            logger.warning(f"Score {score} out of bounds; clamping to [0.0, 1.0]")
            score = max(0.0, min(1.0, score))

        return {"score": score, "reasoning": str(reasoning)}

    except Exception as e:
        logger.error(f"Judge call failed: {e}", exc_info=True)
        return {"score": None, "reasoning": "JUDGE_ERROR"}

def judge_faithfulness(
    question: str,
    answer: str,
    retrieved_chunks: list[str],
    ground_truth_answer: str,
    ground_truth_chunk_ids: list[str],
    retrieved_chunk_ids: list[str],
    client: Groq,
) -> dict:
    context_text = "\n\n---\n\n".join(retrieved_chunks)

    prompt = f"""You are an expert evaluator assessing the FAITHFULNESS of
a generated answer.
Faithfulness measures whether the answer sticks strictly to the retrieved
context with zero hallucination.

QUESTION: {question}

GENERATED ANSWER: {answer}

RETRIEVED CONTEXT:
{context_text}

Evaluate whether every claim, fact, and statement in the GENERATED ANSWER
is directly supported by the RETRIEVED CONTEXT.
- 1.0 = Every claim is directly supported by the context. Zero
hallucination.
- 0.7-0.9 = Minor unsupported inferences or slight embellishments, but core
claims are supported.
- 0.4-0.6 = Some claims are unsupported, or the answer mixes supported
facts with hallucinated details.
- 0.1-0.3 = Most claims are unsupported or the answer contradicts the
context.
- 0.0 = The answer completely ignores or contradicts the context.

```

Output ONLY a JSON object in this exact format:
 {"score": <float between 0.0 and 1.0>, "reasoning": "<detailed explanation>"}

Do not include any markdown formatting, preamble, or text outside the JSON object."""

```
return _call_judge(prompt, client)
```

```
def judge_answer_relevance(
    question: str,
    answer: str,
    retrieved_chunks: list[str],
    ground_truth_answer: str,
    ground_truth_chunk_ids: list[str],
    retrieved_chunk_ids: list[str],
    client: Groq,
) -> dict:
    prompt = f"""You are an expert evaluator assessing ANSWER RELEVANCE.
    Answer relevance measures whether the generated answer actually and
    completely addresses the question asked.
```

```
QUESTION: {question}
```

```
GENERATED ANSWER: {answer}
```

```
GROUND TRUTH ANSWER: {ground_truth_answer}
```

```
Evaluate how well the GENERATED ANSWER addresses the QUESTION.
- 1.0 = The answer fully and accurately addresses every aspect of the
question.
- 0.7-0.9 = The answer addresses the question well but misses minor
nuances.
- 0.4-0.6 = The answer partially addresses the question or misses key
aspects.
- 0.1-0.3 = The answer is tangentially related or largely incomplete.
- 0.0 = The answer is completely irrelevant or does not address the
question.
```

Output ONLY a JSON object in this exact format:
 {"score": <float between 0.0 and 1.0>, "reasoning": "<detailed explanation>"}

Do not include any markdown formatting, preamble, or text outside the JSON object."""

```
return _call_judge(prompt, client)
```

```
def judge_context_precision(
    question: str,
    answer: str,
```

```

    retrieved_chunks: list[str],
    ground_truth_answer: str,
    ground_truth_chunk_ids: list[str],
    retrieved_chunk_ids: list[str],
    client: Groq,
) -> dict:
    context_text = "\n\n---\n\n".join(retrieved_chunks)

    prompt = f"""You are an expert evaluator assessing CONTEXT PRECISION.
Context precision measures whether the retrieved chunks were genuinely
useful and relevant, or if they contained noisy/irrelevant junk.

QUESTION: {question}

RETRIEVED CHUNKS:
{context_text}

RETRIEVED CHUNK IDs: {retrieved_chunk_ids}
GROUND TRUTH CHUNK IDs: {ground_truth_chunk_ids}

Evaluate whether the retrieved chunks are relevant and useful for answering
the question.
- 1.0 = All retrieved chunks are highly relevant and necessary for
answering the question.
- 0.7-0.9 = Most chunks are relevant, but one or two contain minor noise.
- 0.4-0.6 = Roughly half the chunks are relevant; significant noise is
present.
- 0.1-0.3 = Most chunks are irrelevant or useless for the question.
- 0.0 = All retrieved chunks are completely irrelevant junk.

Output ONLY a JSON object in this exact format:
{"score": <float between 0.0 and 1.0>, "reasoning": "<detailed
explanation>"}}

Do not include any markdown formatting, preamble, or text outside the JSON
object."""

    return _call_judge(prompt, client)

def judge_context_recall(
    question: str,
    answer: str,
    retrieved_chunks: list[str],
    ground_truth_answer: str,
    ground_truth_chunk_ids: list[str],
    retrieved_chunk_ids: list[str],
    client: Groq,
) -> dict:
    context_text = "\n\n---\n\n".join(retrieved_chunks)

    prompt = f"""You are an expert evaluator assessing CONTEXT RECALL.
Context recall measures whether the retrieval step surfaced ALL the chunks
needed to fully answer the question.

```

QUESTION: {question}

RETRIEVED CHUNKS:
{context_text}

RETRIEVED CHUNK IDs: {retrieved_chunk_ids}
GROUND TRUTH CHUNK IDs: {ground_truth_chunk_ids}

Evaluate whether the retrieved chunks contain ALL the information necessary to answer the question fully.

Compare the retrieved chunk IDs against the ground truth chunk IDs, and assess the content coverage.

- 1.0 = All ground truth chunks were retrieved; the retrieved content is fully sufficient.
- 0.7-0.9 = Most critical chunks were retrieved; minor information may be missing but the answer is still largely achievable.
- 0.4-0.6 = Some important chunks are missing; the retrieved content is incomplete.
- 0.1-0.3 = Most critical chunks are missing; the retrieved content is insufficient.
- 0.0 = None of the necessary chunks were retrieved.

Output ONLY a JSON object in this exact format:

```
{{"score": <float between 0.0 and 1.0>, "reasoning": "<detailed explanation>"}}
```

Do not include any markdown formatting, preamble, or text outside the JSON object. """

```
return _call_judge(prompt, client)
```

F. eval/run_eval.py

```
# eval/run_eval.py
import json
import os
import time
from pathlib import Path

import chromadb
import matplotlib.pyplot as plt
import pandas as pd
from groq import Groq
from tqdm import tqdm

from config import settings
from eval.judges import (
    judge_answer_relevance,
    judge_context_precision,
    judge_context_recall,
```

```
        judge_faithfulness,
    )
    from src.rag_bot import get_rag_response
    from src.utils import load_jsonl, setup_logger

    logger = setup_logger(__name__)

    def load_test_set(path: str) -> list[dict]:
        logger.info(f"Loading test set from {path}")
        data = load_jsonl(path)
        logger.info(f"Loaded {len(data)} questions from {path}")
        return data

    def run_single_evaluation(item: dict, collection, client) -> dict:
        question_id = item["question_id"]
        question = item["question"]
        ground_truth_answer = item["ground_truth_answer"]
        ground_truth_chunk_ids = item["ground_truth_chunk_ids"]

        logger.info(f"Evaluating {question_id}: {question[:80]}...")

        rag_result = get_rag_response(question, collection, client)
        answer = rag_result["answer"]
        retrieved_chunks = rag_result["retrieved_chunks"]
        retrieved_chunk_ids = rag_result["retrieved_chunk_ids"]

        faithfulness = judge_faithfulness(
            question,
            answer,
            retrieved_chunks,
            ground_truth_answer,
            ground_truth_chunk_ids,
            retrieved_chunk_ids,
            client,
        )
        answer_relevance = judge_answer_relevance(
            question,
            answer,
            retrieved_chunks,
            ground_truth_answer,
            ground_truth_chunk_ids,
            retrieved_chunk_ids,
            client,
        )
        context_precision = judge_context_precision(
            question,
            answer,
            retrieved_chunks,
            ground_truth_answer,
            ground_truth_chunk_ids,
            retrieved_chunk_ids,
            client,
```

```

    )
    context_recall = judge_context_recall(
        question,
        answer,
        retrieved_chunks,
        ground_truth_answer,
        ground_truth_chunk_ids,
        retrieved_chunk_ids,
        client,
    )

    scores = [
        faithfulness["score"],
        answer_relevance["score"],
        context_precision["score"],
        context_recall["score"],
    ]
    valid_scores = [s for s in scores if s is not None]
    avg_score = (
        round(sum(valid_scores) / len(valid_scores), 4) if valid_scores
    else None
    )

    result = {
        "question_id": question_id,
        "question": question,
        "answer": answer,
        "retrieved_chunk_ids": json.dumps(retrieved_chunk_ids),
        "faithfulness_score": faithfulness["score"],
        "faithfulness_reasoning": faithfulness["reasoning"],
        "answer_relevance_score": answer_relevance["score"],
        "answer_relevance_reasoning": answer_relevance["reasoning"],
        "context_precision_score": context_precision["score"],
        "context_precision_reasoning": context_precision["reasoning"],
        "context_recall_score": context_recall["score"],
        "context_recall_reasoning": context_recall["reasoning"],
        "avg_score": avg_score,
    }

    return result

def save_results_csv(results: list[dict], path: str) -> None:
    df = pd.DataFrame(results)
    score_cols = [
        "faithfulness_score",
        "answer_relevance_score",
        "context_precision_score",
        "context_recall_score",
        "avg_score",
    ]
    for col in score_cols:
        if col in df.columns:
            df[col] = df[col].apply(

```

```

        lambda x: round(x, 4) if pd.notna(x) and x is not None else
x
    )

    Path(path).parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    logger.info(f"Saved results CSV to {path} ({len(df)} rows)")

def _df_to_markdown(df: pd.DataFrame) -> str:
    lines = []
    headers = "| " + " | ".join(df.columns) + " |"
    separators = "| " + " | ".join(["---"] * len(df.columns)) + " |"
    lines.append(headers)
    lines.append(separators)
    for _, row in df.iterrows():
        row_str = (
            "| "
            + " | ".join(str(v) if v is not None else "N/A" for v in
row.values)
            + " |"
        )
        lines.append(row_str)
    return "\n".join(lines)

def generate_eval_report(results_df: pd.DataFrame, output_path: str) ->
None:
    logger.info("Generating evaluation report...")

    score_cols = [
        "faithfulness_score",
        "answer_relevance_score",
        "context_precision_score",
        "context_recall_score",
    ]

    stats = []
    for col in score_cols:
        valid = results_df[col].dropna()
        stats.append(
            {
                "Metric": col.replace("_score", "").replace("_", "
").title(),
                "Mean": round(valid.mean(), 4) if len(valid) > 0 else None,
                "Min": round(valid.min(), 4) if len(valid) > 0 else None,
                "Max": round(valid.max(), 4) if len(valid) > 0 else None,
                "Std Dev": round(valid.std(), 4) if len(valid) > 0 else
None,
                "Valid Count": len(valid),
            }
        )
    stats_df = pd.DataFrame(stats)

```

```

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()
colors = ["#2ecc71", "#3498db", "#e74c3c", "#f39c12"]

for idx, col in enumerate(score_cols):
    ax = axes[idx]
    valid = results_df[col].dropna()
    if len(valid) > 0:
        ax.hist(
            valid,
            bins=10,
            range=(0, 1),
            color=colors[idx],
            edgecolor="black",
            alpha=0.7,
        )
        ax.set_title(col.replace("_score", "").replace("_", "
").title())
        ax.set_xlabel("Score")
        ax.set_ylabel("Frequency")
        ax.set_xlim(0, 1)
    else:
        ax.text(
            0.5,
            0.5,
            "No valid scores",
            ha="center",
            va="center",
            transform=ax.transAxes,
        )
        ax.set_title(col.replace("_score", "").replace("_", "
").title())

plt.tight_layout()
hist_path = Path(output_path).parent / "score_distribution.png"
plt.savefig(hist_path, dpi=150, bbox_inches="tight")
plt.close()
logger.info(f"Saved histogram to {hist_path}")

if results_df["avg_score"].notna().sum() == 0:
    logger.warning("No valid average scores; best/worst case selection
may be arbitrary.")
    top3 = results_df.head(3)
    bottom3 = results_df.tail(3)
else:
    top3 = results_df.sort_values("avg_score", ascending=False,
na_position="last").head(3)
    bottom3 = results_df.sort_values("avg_score", ascending=True,
na_position="first").head(3)

lines = []
lines.append("# RAG Evaluation Report")
lines.append(f"\nGenerated: {pd.Timestamp.now().strftime('%Y-%m-%d
%H:%M:%S')}")

```

```

lines.append(f"\nTotal Questions Evaluated: {len(results_df)}")
lines.append("\n---\n")

lines.append("## Aggregate Metrics\n")
lines.append(_df_to_markdown(stats_df))
lines.append("\n")

lines.append("## Score Distribution\n")
lines.append("![Score Distribution](score_distribution.png)")
lines.append("\n")

lines.append("## Top 3 Best Answers (by Average Score)\n")
for _, row in top3.iterrows():
    lines.append(f"### {row['question_id']}: {row['question']}\n")
    lines.append(f"**Answer:** {row['answer']}\n")
    lines.append(f"**Retrieved Chunks:**\n")
    lines.append(f"{row['retrieved_chunk_ids']}\n")
    lines.append(f"**Avg Score:** {row['avg_score']}\n")
    lines.append(
        f"- **Faithfulness ({row['faithfulness_score']}):**\n")
    lines.append(f"{row['faithfulness_reasoning']}\n")
    lines.append(
        f"- **Answer Relevance ({row['answer_relevance_score']}):**\n")
    lines.append(f"{row['answer_relevance_reasoning']}\n")
    lines.append(
        f"- **Context Precision ({row['context_precision_score']}):**\n")
    lines.append(f"{row['context_precision_reasoning']}\n")
    lines.append(
        f"- **Context Recall ({row['context_recall_score']}):**\n")
    lines.append(f"{row['context_recall_reasoning']}\n")
    lines.append("\n---\n")

lines.append("## Bottom 3 Worst Answers (by Average Score)\n")
for _, row in bottom3.iterrows():
    lines.append(f"### {row['question_id']}: {row['question']}\n")
    lines.append(f"**Answer:** {row['answer']}\n")
    lines.append(f"**Retrieved Chunks:**\n")
    lines.append(f"{row['retrieved_chunk_ids']}\n")
    lines.append(f"**Avg Score:** {row['avg_score']}\n")
    lines.append(
        f"- **Faithfulness ({row['faithfulness_score']}):**\n")
    lines.append(f"{row['faithfulness_reasoning']}\n")
    lines.append(
        f"- **Answer Relevance ({row['answer_relevance_score']}):**\n")
    lines.append(f"{row['answer_relevance_reasoning']}\n")
    lines.append(
        f"- **Context Precision ({row['context_precision_score']}):**\n")
    lines.append(f"{row['context_precision_reasoning']}\n")

```

```

    )
    lines.append(
        f"-- **Context Recall ({row['context_recall_score']}):**\n"
        {row['context_recall_reasoning']}\n"
    )

    metric_scores = {
        "Faithfulness": row["faithfulness_score"],
        "Answer Relevance": row["answer_relevance_score"],
        "Context Precision": row["context_precision_score"],
        "Context Recall": row["context_recall_score"],
    }
    valid_metrics = {k: v for k, v in metric_scores.items() if v is not
None}
    if valid_metrics:
        lowest_metric = min(valid_metrics, key=valid_metrics.get)
        lowest_score = valid_metrics[lowest_metric]
        lines.append(
            f"\n**Root Cause Analysis:** The lowest score was **\n"
            {lowest_metric}** ({lowest_score}). "
        )
        if lowest_metric == "Faithfulness":
            lines.append(
                "The answer likely hallucinated or contradicted the\n"
                retrieved context. Review the chunking strategy and ensure the generator\n"
                prompt strictly grounds the model.\n"
            )
        elif lowest_metric == "Answer Relevance":
            lines.append(
                "The answer failed to address the actual question. The\n"
                retrieval may have fetched off-topic chunks, or the generator may have\n"
                misinterpreted the question.\n"
            )
        elif lowest_metric == "Context Precision":
            lines.append(
                "The retriever returned noisy or irrelevant chunks.\n"
                Consider tuning the embedding model, adjusting top_k, or filtering chunks\n"
                by similarity threshold.\n"
            )
        elif lowest_metric == "Context Recall":
            lines.append(
                "The retriever missed critical chunks needed to answer\n"
                the question. The corpus may lack coverage, or the embedding similarity\n"
                search failed to surface the correct chunks.\n"
            )
        lines.append("\n---\n")

    lines.append("## Prioritized Improvement Roadmap\n")
    lines.append(
        "Based on aggregate metric scores, ranked from lowest-performing to\n"
        highest:\n"
    )
    metric_means = {
        "Faithfulness": stats_df[stats_df["Metric"] == "Faithfulness"]

```

```

["Mean"].values[0],
    "Answer Relevance": stats_df[stats_df["Metric"] == "Answer
Relevance"]["Mean"].values[0],
    "Context Precision": stats_df[stats_df["Metric"] == "Context
Precision"]["Mean"].values[0],
    "Context Recall": stats_df[stats_df["Metric"] == "Context Recall"]
["Mean"].values[0],
}
sorted_metrics = sorted(
    [(k, v) for k, v in metric_means.items() if v is not None],
    key=lambda x: x[1],
)

for rank, (metric, mean) in enumerate(sorted_metrics, 1):
    lines.append(f"{rank}. **{metric}** (Mean: {mean}) - ")
    if metric == "Faithfulness":
        lines.append(
            "Strengthen the generator system prompt with stricter
grounding instructions. Add a post-generation fact-checking layer against
retrieved chunks.\n"
        )
    elif metric == "Answer Relevance":
        lines.append(
            "Improve query understanding by adding query rewriting or
expansion. Ensure the generator is explicitly instructed to answer the
exact question asked.\n"
        )
    elif metric == "Context Precision":
        lines.append(
            "Implement a similarity threshold filter to drop low-
relevance chunks. Consider re-ranking retrieved chunks with a cross-encoder
before passing to the generator.\n"
        )
    elif metric == "Context Recall":
        lines.append(
            "Increase top_k or implement hybrid search (dense +
keyword). Audit the corpus for coverage gaps and add missing documents.\n"
        )

    report_text = "\n".join(lines)
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(report_text)
    logger.info(f"Saved evaluation report to {output_path}")

def main():
    logger.info("=== Evaluation Pipeline Started ===")
    start_time = time.time()

    from dotenv import load_dotenv

    load_dotenv()
    api_key = os.getenv("GROQ_API_KEY")

```

```

if not api_key:
    logger.error("GROQ_API_KEY not found in environment")
    raise ValueError("GROQ_API_KEY not found")

client = Groq(api_key=api_key)

chroma_client =
chromadb.PersistentClient(path=settings.CHROMA_PERSIST_DIR)
collection = chroma_client.get_or_create_collection(name="rag_corpus")

test_set = load_test_set(settings.EVAL_TEST_SET_PATH)

results = []
full_success = 0
partial_failure = 0

for item in tqdm(test_set, desc="Evaluating"):
    try:
        result = run_single_evaluation(item, collection, client)
        results.append(result)

        scores = [
            result["faithfulness_score"],
            result["answer_relevance_score"],
            result["context_precision_score"],
            result["context_recall_score"],
        ]
        if all(s is not None for s in scores):
            full_success += 1
        else:
            partial_failure += 1
    except Exception as e:
        logger.error(
            f"Fatal error evaluating {item.get('question_id',
'UNKNOWN')}: {e}",
            exc_info=True,
        )
        partial_failure += 1
        continue

save_results_csv(results, settings.EVAL_RESULTS_PATH)
results_df = pd.DataFrame(results)
generate_eval_report(results_df, "eval/eval_report.md")

elapsed = time.time() - start_time
logger.info(
    f"=== Evaluation Pipeline Completed in {elapsed:.2f}s ==="
)
logger.info(
    f"Summary: {full_success}/{len(test_set)} questions fully scored,
{partial_failure} had partial failures"
)

```

```
if __name__ == "__main__":  
    main()
```

G. eval/test_set.jsonl

```
# eval/test_set.jsonl  
{  
  "question_id": "q01", "question": "What is the company's policy on remote  
work, and how many days per week are employees allowed to work from home?",  
  "ground_truth_answer": "Employees may work remotely up to 2 days per week,  
subject to manager approval and role suitability. Remote work requests must  
be submitted via the HR portal at least 48 hours in advance.",  
  "ground_truth_chunk_ids": ["chunk_hr_remote_01", "chunk_hr_remote_02"]  
}  
{  
  "question_id": "q02", "question": "How many days of paid annual leave are  
full-time employees entitled to, and does this increase with tenure?",  
  "ground_truth_answer": "Full-time employees receive 20 days of paid annual  
leave per year. After 3 years of continuous service, this increases to 25  
days. Unused leave can be carried over up to a maximum of 5 days into the  
next calendar year.", "ground_truth_chunk_ids": ["chunk_hr_leave_01",  
"chunk_hr_leave_02"]  
}  
{  
  "question_id": "q03", "question": "What is the procedure for requesting  
maternity leave, and how far in advance must the request be submitted?",  
  "ground_truth_answer": "Employees must submit a maternity leave request at  
least 12 weeks before the expected due date. The request must include a  
medical certificate and be routed through the direct manager and HR  
department. Standard maternity leave is 26 weeks at full pay.",  
  "ground_truth_chunk_ids": ["chunk_hr_maternity_01",  
"chunk_hr_maternity_02"]  
}  
{  
  "question_id": "q04", "question": "Does the company's health insurance  
plan cover dependents, and what is the premium contribution split between  
employer and employee?", "ground_truth_answer": "Yes, the health insurance  
plan covers spouses and dependent children. The employer contributes 80% of  
the premium, while the employee contributes the remaining 20% via monthly  
payroll deduction. Dental and vision coverage are included at no additional  
cost.", "ground_truth_chunk_ids": ["chunk_hr_benefits_01",  
"chunk_hr_benefits_02"]  
}  
{  
  "question_id": "q05", "question": "What is the dress code for the  
corporate headquarters, and are there exceptions for client-facing roles?",  
  "ground_truth_answer": "The corporate headquarters maintains a business-  
casual dress code. Employees in client-facing roles (sales, consulting,  
executive relations) are required to wear formal business attire on days  
with external meetings. Casual Fridays allow jeans and polo shirts.",  
  "ground_truth_chunk_ids": ["chunk_hr_dresscode_01",  
"chunk_hr_dresscode_02"]  
}
```

H. tests/test_judges.py

```
# tests/test_judges.py  
from unittest.mock import MagicMock
```

```

from eval.judges import judge_faithfulness

def test_judge_faithfulness_perfect_score():
    mock_client = MagicMock()
    mock_response = MagicMock()
    mock_response.choices = [MagicMock()]
    mock_response.choices[0].message.content = '{"score": 1.0, "reasoning":
    "All claims supported."}'
    mock_client.chat.completions.create.return_value = mock_response

    result = judge_faithfulness(
        question="What is the policy?",
        answer="The policy is X.",
        retrieved_chunks=["The policy is X."],
        ground_truth_answer="The policy is X.",
        ground_truth_chunk_ids=["chunk_1"],
        retrieved_chunk_ids=["chunk_1"],
        client=mock_client,
    )
    assert result["score"] == 1.0
    assert result["reasoning"] == "All claims supported."

def test_judge_faithfulness_api_failure():
    mock_client = MagicMock()
    mock_client.chat.completions.create.side_effect = Exception("API
    Error")

    result = judge_faithfulness(
        question="What is the policy?",
        answer="The policy is X.",
        retrieved_chunks=["The policy is X."],
        ground_truth_answer="The policy is X.",
        ground_truth_chunk_ids=["chunk_1"],
        retrieved_chunk_ids=["chunk_1"],
        client=mock_client,
    )
    assert result["score"] is None
    assert result["reasoning"] == "JUDGE_ERROR"

def test_score_clamping():
    mock_client = MagicMock()
    mock_response = MagicMock()
    mock_response.choices = [MagicMock()]
    mock_response.choices[0].message.content = '{"score": 1.5, "reasoning":
    "Too high"}'
    mock_client.chat.completions.create.return_value = mock_response

    result = judge_faithfulness(
        question="What is the policy?",
        answer="The policy is X.",
        retrieved_chunks=["The policy is X."],

```

```

        ground_truth_answer="The policy is X.",
        ground_truth_chunk_ids=["chunk_1"],
        retrieved_chunk_ids=["chunk_1"],
        client=mock_client,
    )
    assert result["score"] == 1.0

```

I. tests/test_rag_bot.py

```

# tests/test_rag_bot.py
from unittest.mock import MagicMock, patch

from src.rag_bot import generate_answer, retrieve

def test_generate_answer_uses_context_only():
    mock_client = MagicMock()
    mock_response = MagicMock()
    mock_response.choices = [MagicMock()]
    mock_response.choices[0].message.content = "Test answer"
    mock_client.chat.completions.create.return_value = mock_response

    question = "What is the policy?"
    context_chunks = ["The remote work policy allows 2 days per week."]

    result = generate_answer(question, context_chunks, mock_client)

    call_args = mock_client.chat.completions.create.call_args
    messages = call_args[1]["messages"]
    prompt_content = messages[1]["content"]
    assert "The remote work policy allows 2 days per week." in prompt_content
    assert "You are a grounded assistant" in messages[0]["content"]
    assert result == "Test answer"

def test_retrieve_returns_correct_structure():
    mock_collection = MagicMock()
    mock_collection.query.return_value = {
        "documents": ["chunk1", "chunk2"],
        "ids": ["id1", "id2"],
    }

    with patch("src.rag_bot._get_embedder") as mock_get_embedder:
        mock_embedder = MagicMock()
        mock_array = MagicMock()
        mock_array.tolist.return_value = [[0.1, 0.2, 0.3]]
        mock_embedder.encode.return_value = mock_array
        mock_get_embedder.return_value = mock_embedder

        result = retrieve("test query", mock_collection, 2)

```

```

assert isinstance(result, tuple)
assert len(result) == 2
assert isinstance(result[0], list)
assert isinstance(result[1], list)
assert result[0] == ["chunk1", "chunk2"]
assert result[1] == ["id1", "id2"]

```

Section 4: Code Logic & Deep-Dive

4.1 Judge Prompt Engineering Deep-Dive

Faithfulness: The prompt forces the judge to perform a claim-by-claim audit of the generated answer against the retrieved context. The rubric is granular (1.0, 0.7-0.9, 0.4-0.6, 0.1-0.3, 0.0) to prevent the judge from defaulting to binary good/bad judgments. Without this granularity, the judge might conflate minor paraphrasing with outright hallucination, collapsing the signal needed to debug the generator.

Answer Relevance: The prompt explicitly provides the ground-truth answer as a reference point, but instructs the judge to evaluate the *generated* answer against the *question*, not against the ground truth. This guards against the failure mode where a factually correct but off-topic answer receives a high score. If the rubric were vague, the judge might score based on fluency rather than question coverage.

Context Precision: The prompt surfaces both the retrieved chunk IDs and the ground-truth chunk IDs, giving the judge structural anchors to verify relevance. The rubric distinguishes between "minor noise" (0.7-0.9) and "significant noise" (0.4-0.6), which is critical for diagnosing whether the retriever is returning semantically similar but factually irrelevant chunks—a common failure mode in dense retrieval.

Context Recall: The prompt asks the judge to compare ID sets *and* assess content coverage. This dual signal prevents the judge from giving a perfect recall score simply because the retrieved chunks look comprehensive, even if they miss a critical ground-truth chunk. Without explicit ID comparison, the judge might miss structural gaps in retrieval coverage.

4.2 JSON Parsing Fragility & Defense

Calling `json.loads()` directly on an LLM response is inherently fragile because production LLMs frequently wrap JSON in markdown fences (`````` rag-eval-harness/ |—— myenv/ |—— data/ | |—— raw/ # Intern drops .txt corpus files here |—— src/ | |—— **init.py** | |—— ingest.py # Chunking + ChromaDB indexing pipeline | |—— rag_bot.py # RAG retrieval + generation logic | |—— utils.py # Shared utilities (logging setup, token counting) |—— eval/ | |—— test_set.jsonl # 30 questions with ground-truth answers + chunk_ids | |—— judges.py # 4 judge functions (one per metric) | |—— run_eval.py # Master evaluation orchestrator | |—— results.csv # Auto-generated: scores for all 30 questions | |—— eval_report.md # Auto-generated: aggregate analysis + best/worst cases |—— tests/ | |—— test_judges.py | |—— test_rag_bot.py |—— config/ | |—— settings.py # Central config: model names, chunk size, top_k, thresholds |—— .env # GROQ_API_KEY lives here (never commit this) |—— .gitignore |—— requirements.txt |—— output.log # Auto-created: all logs mirrored here |—— guide.md

2.2 One-Shot Bash Setup Script

```
```bash
setup.sh
#!/bin/bash
set -e

PROJECT_NAME="rag-eval-harness"
echo "Setting up $PROJECT_NAME..."

Create directory structure
mkdir -p data/raw
mkdir -p src
mkdir -p eval
mkdir -p tests
mkdir -p config

Create empty Python package and source files
touch src/__init__.py
touch src/ingest.py
touch src/rag_bot.py
touch src/utils.py
touch eval/judges.py
touch eval/run_eval.py
touch eval/test_set.jsonl
touch eval/results.csv
touch eval/eval_report.md
touch tests/test_judges.py
touch tests/test_rag_bot.py
touch config/settings.py
touch requirements.txt
touch guide.md
touch output.log

Create virtual environment (name: myenv, NOT venv)
python3 -m venv myenv

Create .gitignore
cat > .gitignore << 'EOF'
myenv/
.env
output.log
__pycache__/
*.pyc
chroma_db/
EOF

Create .env.example
cat > .env.example << 'EOF'
GROQ_API_KEY=your_key_here
EOF
```

```
Create requirements.txt
cat > requirements.txt << 'EOF'
groq>=0.9.0
chromadb>=0.5.0
sentence-transformers>=3.0.0
pandas>=2.0.0
matplotlib>=3.8.0
jsonlines>=4.0.0
python-dotenv>=1.0.0
tqdm>=4.66.0
pytest>=8.0.0
EOF

Install dependencies inside the virtual environment
source myenv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

echo "Setup complete. Activate the environment with: source
myenv/bin/activate"
```

---

## Section 3: Production-Ready Implementation Code

### A. `config/settings.py`

```
config/settings.py
from typing import Final

GROQ_MODEL: Final[str] = "llama-3.3-70b-versatile"
EMBEDDING_MODEL: Final[str] = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE: Final[int] = 512
CHUNK_OVERLAP: Final[int] = 128
TOP_K_RETRIEVAL: Final[int] = 5
CHROMA_PERSIST_DIR: Final[str] = "chroma_db"
EVAL_TEST_SET_PATH: Final[str] = "eval/test_set.jsonl"
EVAL_RESULTS_PATH: Final[str] = "eval/results.csv"
LOG_FILE_PATH: Final[str] = "output.log"
SCORE_THRESHOLD: Final[float] = 0.6
```

### B. `src/utils.py`

```
src/utils.py
import json
import logging
import sys
from pathlib import Path
```

```
from config.settings import LOG_FILE_PATH

def setup_logger(name: str) -> logging.Logger:
 logger = logging.getLogger(name)
 logger.setLevel(logging.INFO)

 if logger.hasHandlers():
 logger.handlers.clear()

 formatter = logging.Formatter(
 "%(asctime)s | %(levelname)s | %(message)s",
 datefmt="%Y-%m-%d %H:%M:%S",
)

 stream_handler = logging.StreamHandler(sys.stdout)
 stream_handler.setLevel(logging.INFO)
 stream_handler.setFormatter(formatter)
 logger.addHandler(stream_handler)

 file_handler = logging.FileHandler(LOG_FILE_PATH, mode="a")
 file_handler.setLevel(logging.INFO)
 file_handler.setFormatter(formatter)
 logger.addHandler(file_handler)

 return logger

def count_tokens(text: str) -> int:
 # Heuristic: 1 word ≈ 1.3 tokens on average for English text.
 # This is a fast approximation. For production, use tiktoken or the
 # model's tokenizer.
 return int(len(text.split()) * 1.3)

def load_jsonl(path: str) -> list[dict]:
 logger = setup_logger(__name__)
 data = []
 try:
 with open(path, "r", encoding="utf-8") as f:
 for line_num, line in enumerate(f, 1):
 line = line.strip()
 if not line:
 continue
 try:
 data.append(json.loads(line))
 except json.JSONDecodeError as e:
 logger.error(f"Invalid JSON on line {line_num} in {path}: {e}")
 except FileNotFoundError:
 logger.error(f"File not found: {path}")
 raise
 except Exception as e:
 logger.error(f"Error reading {path}: {e}")
```

```

 raise
 return data

def save_jsonl(data: list[dict], path: str) -> None:
 logger = setup_logger(__name__)
 try:
 Path(path).parent.mkdir(parents=True, exist_ok=True)
 with open(path, "w", encoding="utf-8") as f:
 for item in data:
 f.write(json.dumps(item, ensure_ascii=False) + "\n")
 logger.info(f"Saved {len(data)} records to {path}")
 except Exception as e:
 logger.error(f"Error writing to {path}: {e}")
 raise

```

### C. src/ingest.py

```

src/ingest.py
import hashlib
import time
from pathlib import Path

import chromadb
from sentence_transformers import SentenceTransformer
from tqdm import tqdm

from config import settings
from src.utils import setup_logger, count_tokens

logger = setup_logger(__name__)

def load_corpus(data_dir: str) -> list[str]:
 texts = []
 data_path = Path(data_dir)
 if not data_path.exists():
 logger.error(f"Data directory not found: {data_dir}")
 raise FileNotFoundError(f"Data directory not found: {data_dir}")

 txt_files = sorted(data_path.glob("*.txt"))
 total_chars = 0

 for txt_file in txt_files:
 try:
 with open(txt_file, "r", encoding="utf-8") as f:
 text = f.read()
 texts.append(text)
 total_chars += len(text)
 logger.info(f"Loaded {txt_file.name}: {len(text)} chars")
 except Exception as e:

```

```

 logger.error(f"Failed to read {txt_file.name}: {e}")

 logger.info(f"Loaded {len(texts)} files, {total_chars} total
characters")
 return texts

def chunk_text(text: str, chunk_size: int, overlap: int) -> list[str]:
 words = text.split()
 chunks = []
 if not words:
 return chunks

 start = 0
 while start < len(words):
 end = min(start + chunk_size, len(words))
 chunk_words = words[start:end]
 chunks.append(" ".join(chunk_words))
 if end == len(words):
 break
 start += chunk_size - overlap

 return chunks

def generate_chunk_id(text: str, index: int) -> str:
 content = f"{index}:{text[:50]}"
 return hashlib.md5(content.encode("utf-8")).hexdigest()

def build_chroma_index(chunks: list[str], chunk_ids: list[str]) ->
chromadb.Collection:
 client = chromadb.PersistentClient(path=settings.CHROMA_PERSIST_DIR)
 collection = client.get_or_create_collection(name="rag_corpus")

 embedder = SentenceTransformer(settings.EMBEDDING_MODEL)

 batch_size = 100
 total_batches = (len(chunks) + batch_size - 1) // batch_size
 for i in tqdm(range(0, len(chunks), batch_size), desc="Embedding &
indexing"):
 batch_chunks = chunks[i : i + batch_size]
 batch_ids = chunk_ids[i : i + batch_size]
 embeddings = embedder.encode(batch_chunks,
show_progress_bar=False).tolist()
 collection.upsert(ids=batch_ids, embeddings=embeddings,
documents=batch_chunks)
 logger.info(f"Indexed batch {i // batch_size + 1}/{total_batches}")

 logger.info(f"Indexed {len(chunks)} chunks into ChromaDB collection
'rag_corpus'")
 return collection

```

```
def main():
 logger.info("=== Ingestion Pipeline Started ===")
 start = time.time()

 raw_texts = load_corpus("data/raw")
 all_chunks = []
 all_chunk_ids = []

 for doc_idx, text in enumerate(raw_texts):
 doc_chunks = chunk_text(text, settings.CHUNK_SIZE,
 settings.CHUNK_OVERLAP)
 base_idx = len(all_chunks)
 for chunk_idx, chunk in enumerate(doc_chunks):
 global_idx = base_idx + chunk_idx
 chunk_id = generate_chunk_id(chunk, global_idx)
 all_chunks.append(chunk)
 all_chunk_ids.append(chunk_id)
 logger.info(f"Document {doc_idx}: chunked into {len(doc_chunks)}
chunks")

 collection = build_chroma_index(all_chunks, all_chunk_ids)
 elapsed = time.time() - start
 logger.info(f"=== Ingestion Pipeline Completed in {elapsed:.2f}s ===")

if __name__ == "__main__":
 main()
```

#### D. src/rag\_bot.py

```
src/rag_bot.py
import chromadb
from groq import Groq
from sentence_transformers import SentenceTransformer

from config import settings
from src.utils import setup_logger

logger = setup_logger(__name__)

_embedder = None

def _get_embedder():
 global _embedder
 if _embedder is None:
 _embedder = SentenceTransformer(settings.EMBEDDING_MODEL)
 return _embedder

def retrieve(
```

```

 query: str, collection: chromadb.Collection, top_k: int
) -> tuple[list[str], list[str]]:
 embedder = _get_embedder()
 query_embedding = embedder.encode([query],
show_progress_bar=False).tolist()[0]

 results = collection.query(
 query_embeddings=[query_embedding],
 n_results=top_k,
 include=["documents"],
)

 documents = results["documents"][0] if results["documents"] else []
 ids = results["ids"][0] if results["ids"] else []

 logger.info(f"Retrieved {len(documents)} chunks for query")
 return documents, ids

def generate_answer(question: str, context_chunks: list[str], client: Groq)
-> str:
 context_text = "\n\n---\n\n".join(context_chunks)

 system_prompt = (
 "You are a grounded assistant. Answer ONLY using the provided\n\ncontext. "
 "If the context does not contain the answer, say: 'I cannot answer\n\nthis from the provided context.' "
 "Do not use any outside knowledge."
)

 user_prompt = f"Context:\n{context_text}\n\nQuestion:\n{question}\n\nAnswer:"

 try:
 response = client.chat.completions.create(
 model=settings.GROQ_MODEL,
 messages=[
 {"role": "system", "content": system_prompt},
 {"role": "user", "content": user_prompt},
],
 temperature=0.0,
 max_tokens=1024,
)
 answer = response.choices[0].message.content.strip()
 logger.info(f"Generated answer ({len(answer)} chars)")
 return answer
 except Exception as e:
 logger.error(f"Groq API error during generation: {e}",
exc_info=True)
 return "ERROR: Generation failed."

def get_rag_response(

```

```

 question: str, collection: chromadb.Collection, client: Groq
) -> dict:
 try:
 retrieved_chunks, retrieved_chunk_ids = retrieve(
 question, collection, settings.TOP_K_RETRIEVAL
)
 answer = generate_answer(question, retrieved_chunks, client)
 return {
 "question": question,
 "answer": answer,
 "retrieved_chunks": retrieved_chunks,
 "retrieved_chunk_ids": retrieved_chunk_ids,
 }
 except Exception as e:
 logger.error(
 f"RAG pipeline failed for question '{question[:50]}...': {e}",
 exc_info=True,
)
 return {
 "question": question,
 "answer": "ERROR: RAG pipeline failed.",
 "retrieved_chunks": [],
 "retrieved_chunk_ids": [],
 }

```

## E. eval/judges.py

```

eval/judges.py
import json
import re

from groq import Groq

from config import settings
from src.utils import setup_logger

logger = setup_logger(__name__)

def _parse_judge_response(response_text: str) -> dict:
 """
 Defensive JSON parser for LLM judge outputs.
 Strips markdown fences, preamble, and extracts the first JSON object.
 """
 text = response_text.strip()

 if text.startswith("```json"):
 text = text[7:]
 elif text.startswith("```"):
 text = text[3:]
 if text.endswith("```"):

```

```

 text = text[:-3]
 text = text.strip()

 try:
 return json.loads(text)
 except json.JSONDecodeError:
 pass

 match = re.search(r"\{.*\}", text, re.DOTALL)
 if match:
 try:
 return json.loads(match.group(0))
 except json.JSONDecodeError:
 pass

 raise ValueError(f"Could not extract valid JSON from response:
{response_text[:200]}")

def _call_judge(prompt: str, client: Groq) -> dict:
 try:
 response = client.chat.completions.create(
 model=settings.GROQ_MODEL,
 messages=[
 {
 "role": "system",
 "content": "You are an expert evaluator. Output ONLY
valid JSON.",
 },
 {"role": "user", "content": prompt},
],
 temperature=0.0,
 max_tokens=512,
)
 raw = response.choices[0].message.content.strip()
 parsed = _parse_judge_response(raw)

 score = parsed.get("score")
 reasoning = parsed.get("reasoning", "No reasoning provided.")

 if score is None:
 raise ValueError("Missing 'score' key in judge response")

 try:
 score = float(score)
 except (ValueError, TypeError):
 raise ValueError(f"Score is not a float: {score}")

 if not (0.0 <= score <= 1.0):
 logger.warning(f"Score {score} out of bounds; clamping to [0.0,
1.0]")
 score = max(0.0, min(1.0, score))

 return {"score": score, "reasoning": str(reasoning)}

```

```

except Exception as e:
 logger.error(f"Judge call failed: {e}", exc_info=True)
 return {"score": None, "reasoning": "JUDGE_ERROR"}

def judge_faithfulness(
 question: str,
 answer: str,
 retrieved_chunks: list[str],
 ground_truth_answer: str,
 ground_truth_chunk_ids: list[str],
 retrieved_chunk_ids: list[str],
 client: Groq,
) -> dict:
 context_text = "\n\n---\n\n".join(retrieved_chunks)

 prompt = f"""You are an expert evaluator assessing the FAITHFULNESS of
a generated answer.
Faithfulness measures whether the answer sticks strictly to the retrieved
context with zero hallucination.

QUESTION: {question}

GENERATED ANSWER: {answer}

RETRIEVED CONTEXT:
{context_text}

Evaluate whether every claim, fact, and statement in the GENERATED ANSWER
is directly supported by the RETRIEVED CONTEXT.
- 1.0 = Every claim is directly supported by the context. Zero
hallucination.
- 0.7-0.9 = Minor unsupported inferences or slight embellishments, but core
claims are supported.
- 0.4-0.6 = Some claims are unsupported, or the answer mixes supported
facts with hallucinated details.
- 0.1-0.3 = Most claims are unsupported or the answer contradicts the
context.
- 0.0 = The answer completely ignores or contradicts the context.

Output ONLY a JSON object in this exact format:
{"score": <float between 0.0 and 1.0>, "reasoning": "<detailed
explanation>"}}

Do not include any markdown formatting, preamble, or text outside the JSON
object."""

 return _call_judge(prompt, client)

def judge_answer_relevance(
 question: str,
 answer: str,

```

```

 retrieved_chunks: list[str],
 ground_truth_answer: str,
 ground_truth_chunk_ids: list[str],
 retrieved_chunk_ids: list[str],
 client: Groq,
) -> dict:
 prompt = f"""You are an expert evaluator assessing ANSWER RELEVANCE.
 Answer relevance measures whether the generated answer actually and
 completely addresses the question asked.

 QUESTION: {question}

 GENERATED ANSWER: {answer}

 GROUND TRUTH ANSWER: {ground_truth_answer}

 Evaluate how well the GENERATED ANSWER addresses the QUESTION.
 - 1.0 = The answer fully and accurately addresses every aspect of the
 question.
 - 0.7-0.9 = The answer addresses the question well but misses minor
 nuances.
 - 0.4-0.6 = The answer partially addresses the question or misses key
 aspects.
 - 0.1-0.3 = The answer is tangentially related or largely incomplete.
 - 0.0 = The answer is completely irrelevant or does not address the
 question.

 Output ONLY a JSON object in this exact format:
 {"score": <float between 0.0 and 1.0>, "reasoning": "<detailed
 explanation>"}

 Do not include any markdown formatting, preamble, or text outside the JSON
 object."""

 return _call_judge(prompt, client)

def judge_context_precision(
 question: str,
 answer: str,
 retrieved_chunks: list[str],
 ground_truth_answer: str,
 ground_truth_chunk_ids: list[str],
 retrieved_chunk_ids: list[str],
 client: Groq,
) -> dict:
 context_text = "\n\n---\n\n".join(retrieved_chunks)

 prompt = f"""You are an expert evaluator assessing CONTEXT PRECISION.
 Context precision measures whether the retrieved chunks were genuinely
 useful and relevant, or if they contained noisy/irrelevant junk.

 QUESTION: {question}

```

RETRIEVED CHUNKS:

{context\_text}

RETRIEVED CHUNK IDs: {retrieved\_chunk\_ids}

GROUND TRUTH CHUNK IDs: {ground\_truth\_chunk\_ids}

Evaluate whether the retrieved chunks are relevant and useful for answering the question.

- 1.0 = All retrieved chunks are highly relevant and necessary for answering the question.
- 0.7-0.9 = Most chunks are relevant, but one or two contain minor noise.
- 0.4-0.6 = Roughly half the chunks are relevant; significant noise is present.
- 0.1-0.3 = Most chunks are irrelevant or useless for the question.
- 0.0 = All retrieved chunks are completely irrelevant junk.

Output ONLY a JSON object in this exact format:

```
{{"score": <float between 0.0 and 1.0>, "reasoning": "<detailed explanation>"}}
```

Do not include any markdown formatting, preamble, or text outside the JSON object.""""

```
 return _call_judge(prompt, client)
```

```
def judge_context_recall(
```

```
 question: str,
```

```
 answer: str,
```

```
 retrieved_chunks: list[str],
```

```
 ground_truth_answer: str,
```

```
 ground_truth_chunk_ids: list[str],
```

```
 retrieved_chunk_ids: list[str],
```

```
 client: Groq,
```

```
) -> dict:
```

```
 context_text = "\n\n---\n\n".join(retrieved_chunks)
```

```
 prompt = f"""You are an expert evaluator assessing CONTEXT RECALL.
```

```
Context recall measures whether the retrieval step surfaced ALL the chunks
needed to fully answer the question.
```

```
QUESTION: {question}
```

```
RETRIEVED CHUNKS:
```

```
{context_text}
```

```
RETRIEVED CHUNK IDs: {retrieved_chunk_ids}
```

```
GROUND TRUTH CHUNK IDs: {ground_truth_chunk_ids}
```

Evaluate whether the retrieved chunks contain ALL the information necessary to answer the question fully.

Compare the retrieved chunk IDs against the ground truth chunk IDs, and assess the content coverage.

- 1.0 = All ground truth chunks were retrieved; the retrieved content is

fully sufficient.

- 0.7–0.9 = Most critical chunks were retrieved; minor information may be missing but the answer is still largely achievable.
- 0.4–0.6 = Some important chunks are missing; the retrieved content is incomplete.
- 0.1–0.3 = Most critical chunks are missing; the retrieved content is insufficient.
- 0.0 = None of the necessary chunks were retrieved.

Output ONLY a JSON object in this exact format:

```
{{"score": <float between 0.0 and 1.0>, "reasoning": "<detailed explanation>"}}
```

Do not include any markdown formatting, preamble, or text outside the JSON object.""""

```
return _call_judge(prompt, client)
```

## F. eval/run\_eval.py

```
eval/run_eval.py
import json
import os
import time
from pathlib import Path

import chromadb
import matplotlib.pyplot as plt
import pandas as pd
from groq import Groq
from tqdm import tqdm

from config import settings
from eval.judges import (
 judge_answer_relevance,
 judge_context_precision,
 judge_context_recall,
 judge_faithfulness,
)
from src.rag_bot import get_rag_response
from src.utils import load_jsonl, setup_logger

logger = setup_logger(__name__)

def load_test_set(path: str) -> list[dict]:
 logger.info(f"Loading test set from {path}")
 data = load_jsonl(path)
 logger.info(f"Loaded {len(data)} questions from {path}")
 return data
```

```
def run_single_evaluation(item: dict, collection, client) -> dict:
 question_id = item["question_id"]
 question = item["question"]
 ground_truth_answer = item["ground_truth_answer"]
 ground_truth_chunk_ids = item["ground_truth_chunk_ids"]

 logger.info(f"Evaluating {question_id}: {question[:80]}...")

 rag_result = get_rag_response(question, collection, client)
 answer = rag_result["answer"]
 retrieved_chunks = rag_result["retrieved_chunks"]
 retrieved_chunk_ids = rag_result["retrieved_chunk_ids"]

 faithfulness = judge_faithfulness(
 question,
 answer,
 retrieved_chunks,
 ground_truth_answer,
 ground_truth_chunk_ids,
 retrieved_chunk_ids,
 client,
)
 answer_relevance = judge_answer_relevance(
 question,
 answer,
 retrieved_chunks,
 ground_truth_answer,
 ground_truth_chunk_ids,
 retrieved_chunk_ids,
 client,
)
 context_precision = judge_context_precision(
 question,
 answer,
 retrieved_chunks,
 ground_truth_answer,
 ground_truth_chunk_ids,
 retrieved_chunk_ids,
 client,
)
 context_recall = judge_context_recall(
 question,
 answer,
 retrieved_chunks,
 ground_truth_answer,
 ground_truth_chunk_ids,
 retrieved_chunk_ids,
 client,
)

 scores = [
 faithfulness["score"],
 answer_relevance["score"],
```

```

 context_precision["score"],
 context_recall["score"],
]
 valid_scores = [s for s in scores if s is not None]
 avg_score = (
 round(sum(valid_scores) / len(valid_scores), 4) if valid_scores
 else None
)

 result = {
 "question_id": question_id,
 "question": question,
 "answer": answer,
 "retrieved_chunk_ids": json.dumps(retrieved_chunk_ids),
 "faithfulness_score": faithfulness["score"],
 "faithfulness_reasoning": faithfulness["reasoning"],
 "answer_relevance_score": answer_relevance["score"],
 "answer_relevance_reasoning": answer_relevance["reasoning"],
 "context_precision_score": context_precision["score"],
 "context_precision_reasoning": context_precision["reasoning"],
 "context_recall_score": context_recall["score"],
 "context_recall_reasoning": context_recall["reasoning"],
 "avg_score": avg_score,
 }

 return result

def save_results_csv(results: list[dict], path: str) -> None:
 df = pd.DataFrame(results)
 score_cols = [
 "faithfulness_score",
 "answer_relevance_score",
 "context_precision_score",
 "context_recall_score",
 "avg_score",
]
 for col in score_cols:
 if col in df.columns:
 df[col] = df[col].apply(
 lambda x: round(x, 4) if pd.notna(x) and x is not None else
x
)

 Path(path).parent.mkdir(parents=True, exist_ok=True)
 df.to_csv(path, index=False)
 logger.info(f"Saved results CSV to {path} ({len(df)} rows)")

def _df_to_markdown(df: pd.DataFrame) -> str:
 lines = []
 headers = "| " + " | ".join(df.columns) + " |"
 separators = "| " + " | ".join(["---"] * len(df.columns)) + " |"
 lines.append(headers)

```

```

lines.append(separators)
for _, row in df.iterrows():
 row_str = (
 "| "
 + " | ".join(str(v) if v is not None else "N/A" for v in
row.values)
 + " |"
)
 lines.append(row_str)
return "\n".join(lines)

def generate_eval_report(results_df: pd.DataFrame, output_path: str) ->
None:
 logger.info("Generating evaluation report...")

 score_cols = [
 "faithfulness_score",
 "answer_relevance_score",
 "context_precision_score",
 "context_recall_score",
]

 stats = []
 for col in score_cols:
 valid = results_df[col].dropna()
 stats.append(
 {
 "Metric": col.replace("_score", "").replace("_", "
").title(),
 "Mean": round(valid.mean(), 4) if len(valid) > 0 else None,
 "Min": round(valid.min(), 4) if len(valid) > 0 else None,
 "Max": round(valid.max(), 4) if len(valid) > 0 else None,
 "Std Dev": round(valid.std(), 4) if len(valid) > 0 else
None,
 "Valid Count": len(valid),
 }
)
 stats_df = pd.DataFrame(stats)

 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
 axes = axes.flatten()
 colors = ["#2ecc71", "#3498db", "#e74c3c", "#f39c12"]

 for idx, col in enumerate(score_cols):
 ax = axes[idx]
 valid = results_df[col].dropna()
 if len(valid) > 0:
 ax.hist(
 valid,
 bins=10,
 range=(0, 1),
 color=colors[idx],
 edgecolor="black",

```

```

 alpha=0.7,
)
 ax.set_title(col.replace("_score", "").replace("_", "
").title())
 ax.set_xlabel("Score")
 ax.set_ylabel("Frequency")
 ax.set_xlim(0, 1)
else:
 ax.text(
 0.5,
 0.5,
 "No valid scores",
 ha="center",
 va="center",
 transform=ax.transAxes,
)
 ax.set_title(col.replace("_score", "").replace("_", "
").title())

plt.tight_layout()
hist_path = Path(output_path).parent / "score_distribution.png"
plt.savefig(hist_path, dpi=150, bbox_inches="tight")
plt.close()
logger.info(f"Saved histogram to {hist_path}")

if results_df["avg_score"].notna().sum() == 0:
 logger.warning("No valid average scores; best/worst case selection
may be arbitrary.")
 top3 = results_df.head(3)
 bottom3 = results_df.tail(3)
else:
 top3 = results_df.sort_values("avg_score", ascending=False,
na_position="last").head(3)
 bottom3 = results_df.sort_values("avg_score", ascending=True,
na_position="first").head(3)

lines = []
lines.append("# RAG Evaluation Report")
lines.append(f"\nGenerated: {pd.Timestamp.now().strftime('%Y-%m-%d
%H:%M:%S')}")
lines.append(f"\nTotal Questions Evaluated: {len(results_df)}")
lines.append("\n---\n")

lines.append("## Aggregate Metrics\n")
lines.append(_df_to_markdown(stats_df))
lines.append("\n")

lines.append("## Score Distribution\n")
lines.append("![Score Distribution](score_distribution.png)")
lines.append("\n")

lines.append("## Top 3 Best Answers (by Average Score)\n")
for _, row in top3.iterrows():
 lines.append(f"### {row['question_id']}: {row['question']}\n")

```

```

 lines.append(f"**Answer:** {row['answer']}\n")
 lines.append(f"**Retrieved Chunks:**\n{row['retrieved_chunk_ids']}\n")
 lines.append(f"**Avg Score:** {row['avg_score']}\n")
 lines.append(
 f"- **Faithfulness ({row['faithfulness_score']}):**\n{row['faithfulness_reasoning']}\n"
)
 lines.append(
 f"- **Answer Relevance ({row['answer_relevance_score']}):**\n{row['answer_relevance_reasoning']}\n"
)
 lines.append(
 f"- **Context Precision ({row['context_precision_score']}):**\n{row['context_precision_reasoning']}\n"
)
 lines.append(
 f"- **Context Recall ({row['context_recall_score']}):**\n{row['context_recall_reasoning']}\n"
)
 lines.append("\n---\n")

 lines.append("## Bottom 3 Worst Answers (by Average Score)\n")
 for _, row in bottom3.iterrows():
 lines.append(f"### {row['question_id']}: {row['question']}\n")
 lines.append(f"**Answer:** {row['answer']}\n")
 lines.append(f"**Retrieved Chunks:**\n{row['retrieved_chunk_ids']}\n")
 lines.append(f"**Avg Score:** {row['avg_score']}\n")
 lines.append(
 f"- **Faithfulness ({row['faithfulness_score']}):**\n{row['faithfulness_reasoning']}\n"
)
 lines.append(
 f"- **Answer Relevance ({row['answer_relevance_score']}):**\n{row['answer_relevance_reasoning']}\n"
)
 lines.append(
 f"- **Context Precision ({row['context_precision_score']}):**\n{row['context_precision_reasoning']}\n"
)
 lines.append(
 f"- **Context Recall ({row['context_recall_score']}):**\n{row['context_recall_reasoning']}\n"
)

 metric_scores = {
 "Faithfulness": row["faithfulness_score"],
 "Answer Relevance": row["answer_relevance_score"],
 "Context Precision": row["context_precision_score"],
 "Context Recall": row["context_recall_score"],
 }
 valid_metrics = {k: v for k, v in metric_scores.items() if v is not
None}

```

```

 if valid_metrics:
 lowest_metric = min(valid_metrics, key=valid_metrics.get)
 lowest_score = valid_metrics[lowest_metric]
 lines.append(
 f"\n**Root Cause Analysis:** The lowest score was **{lowest_metric}** ({lowest_score}). "
)
 if lowest_metric == "Faithfulness":
 lines.append(
 "The answer likely hallucinated or contradicted the retrieved context. Review the chunking strategy and ensure the generator prompt strictly grounds the model.\n"
)
 elif lowest_metric == "Answer Relevance":
 lines.append(
 "The answer failed to address the actual question. The retrieval may have fetched off-topic chunks, or the generator may have misinterpreted the question.\n"
)
 elif lowest_metric == "Context Precision":
 lines.append(
 "The retriever returned noisy or irrelevant chunks. Consider tuning the embedding model, adjusting top_k, or filtering chunks by similarity threshold.\n"
)
 elif lowest_metric == "Context Recall":
 lines.append(
 "The retriever missed critical chunks needed to answer the question. The corpus may lack coverage, or the embedding similarity search failed to surface the correct chunks.\n"
)
 lines.append("\n---\n")

 lines.append("## Prioritized Improvement Roadmap\n")
 lines.append(
 "Based on aggregate metric scores, ranked from lowest-performing to highest:\n"
)
 metric_means = {
 "Faithfulness": stats_df[stats_df["Metric"] == "Faithfulness"]
 ["Mean"].values[0],
 "Answer Relevance": stats_df[stats_df["Metric"] == "Answer Relevance"]
 ["Mean"].values[0],
 "Context Precision": stats_df[stats_df["Metric"] == "Context Precision"]
 ["Mean"].values[0],
 "Context Recall": stats_df[stats_df["Metric"] == "Context Recall"]
 ["Mean"].values[0],
 }
 sorted_metrics = sorted(
 [(k, v) for k, v in metric_means.items() if v is not None],
 key=lambda x: x[1],
)

 for rank, (metric, mean) in enumerate(sorted_metrics, 1):

```

```

 lines.append(f"{rank}. **{metric}** (Mean: {mean}) - ")
 if metric == "Faithfulness":
 lines.append(
 "Strengthen the generator system prompt with stricter grounding instructions. Add a post-generation fact-checking layer against retrieved chunks.\n"
)
 elif metric == "Answer Relevance":
 lines.append(
 "Improve query understanding by adding query rewriting or expansion. Ensure the generator is explicitly instructed to answer the exact question asked.\n"
)
 elif metric == "Context Precision":
 lines.append(
 "Implement a similarity threshold filter to drop low-relevance chunks. Consider re-ranking retrieved chunks with a cross-encoder before passing to the generator.\n"
)
 elif metric == "Context Recall":
 lines.append(
 "Increase top_k or implement hybrid search (dense + keyword). Audit the corpus for coverage gaps and add missing documents.\n"
)

 report_text = "\n".join(lines)
 Path(output_path).parent.mkdir(parents=True, exist_ok=True)
 with open(output_path, "w", encoding="utf-8") as f:
 f.write(report_text)
 logger.info(f"Saved evaluation report to {output_path}")

def main():
 logger.info("=== Evaluation Pipeline Started ===")
 start_time = time.time()

 from dotenv import load_dotenv

 load_dotenv()
 api_key = os.getenv("GROQ_API_KEY")
 if not api_key:
 logger.error("GROQ_API_KEY not found in environment")
 raise ValueError("GROQ_API_KEY not found")

 client = Groq(api_key=api_key)

 chroma_client =
 chromadb.PersistentClient(path=settings.CHROMA_PERSIST_DIR)
 collection = chroma_client.get_or_create_collection(name="rag_corpus")

 test_set = load_test_set(settings.EVAL_TEST_SET_PATH)

 results = []
 full_success = 0

```

```

partial_failure = 0

for item in tqdm(test_set, desc="Evaluating"):
 try:
 result = run_single_evaluation(item, collection, client)
 results.append(result)

 scores = [
 result["faithfulness_score"],
 result["answer_relevance_score"],
 result["context_precision_score"],
 result["context_recall_score"],
]
 if all(s is not None for s in scores):
 full_success += 1
 else:
 partial_failure += 1
 except Exception as e:
 logger.error(
 f"Fatal error evaluating {item.get('question_id',
'UNKNOWN')}: {e}",
 exc_info=True,
)
 partial_failure += 1
 continue

save_results_csv(results, settings.EVAL_RESULTS_PATH)
results_df = pd.DataFrame(results)
generate_eval_report(results_df, "eval/eval_report.md")

elapsed = time.time() - start_time
logger.info(
 f"=== Evaluation Pipeline Completed in {elapsed:.2f}s ==="
)
logger.info(
 f"Summary: {full_success}/{len(test_set)} questions fully scored,
{partial_failure} had partial failures"
)

if __name__ == "__main__":
 main()

```

## G. eval/test\_set.jsonl

```

eval/test_set.jsonl
{"question_id": "q01", "question": "What is the company's policy on remote
work, and how many days per week are employees allowed to work from home?",
"ground_truth_answer": "Employees may work remotely up to 2 days per week,
subject to manager approval and role suitability. Remote work requests must
be submitted via the HR portal at least 48 hours in advance.",

```

```

"ground_truth_chunk_ids": ["chunk_hr_remote_01", "chunk_hr_remote_02"]}
{"question_id": "q02", "question": "How many days of paid annual leave are
full-time employees entitled to, and does this increase with tenure?",
"ground_truth_answer": "Full-time employees receive 20 days of paid annual
leave per year. After 3 years of continuous service, this increases to 25
days. Unused leave can be carried over up to a maximum of 5 days into the
next calendar year.", "ground_truth_chunk_ids": ["chunk_hr_leave_01",
"chunk_hr_leave_02"]}
{"question_id": "q03", "question": "What is the procedure for requesting
maternity leave, and how far in advance must the request be submitted?",
"ground_truth_answer": "Employees must submit a maternity leave request at
least 12 weeks before the expected due date. The request must include a
medical certificate and be routed through the direct manager and HR
department. Standard maternity leave is 26 weeks at full pay.",
"ground_truth_chunk_ids": ["chunk_hr_maternity_01",
"chunk_hr_maternity_02"]}
{"question_id": "q04", "question": "Does the company's health insurance
plan cover dependents, and what is the premium contribution split between
employer and employee?", "ground_truth_answer": "Yes, the health insurance
plan covers spouses and dependent children. The employer contributes 80% of
the premium, while the employee contributes the remaining 20% via monthly
payroll deduction. Dental and vision coverage are included at no additional
cost.", "ground_truth_chunk_ids": ["chunk_hr_benefits_01",
"chunk_hr_benefits_02"]}
{"question_id": "q05", "question": "What is the dress code for the
corporate headquarters, and are there exceptions for client-facing roles?",
"ground_truth_answer": "The corporate headquarters maintains a business-
casual dress code. Employees in client-facing roles (sales, consulting,
executive relations) are required to wear formal business attire on days
with external meetings. Casual Fridays allow jeans and polo shirts.",
"ground_truth_chunk_ids": ["chunk_hr_dresscode_01",
"chunk_hr_dresscode_02"]}

```

## H. tests/test\_judges.py

```

tests/test_judges.py
from unittest.mock import MagicMock

from eval.judges import judge_faithfulness

def test_judge_faithfulness_perfect_score():
 mock_client = MagicMock()
 mock_response = MagicMock()
 mock_response.choices = [MagicMock()]
 mock_response.choices[0].message.content = '{"score": 1.0, "reasoning":
 "All claims supported."}'
 mock_client.chat.completions.create.return_value = mock_response

 result = judge_faithfulness(
 question="What is the policy?",

```

```

 answer="The policy is X.",
 retrieved_chunks=["The policy is X."],
 ground_truth_answer="The policy is X.",
 ground_truth_chunk_ids=["chunk_1"],
 retrieved_chunk_ids=["chunk_1"],
 client=mock_client,
)
 assert result["score"] == 1.0
 assert result["reasoning"] == "All claims supported."

def test_judge_faithfulness_api_failure():
 mock_client = MagicMock()
 mock_client.chat.completions.create.side_effect = Exception("API
Error")

 result = judge_faithfulness(
 question="What is the policy?",
 answer="The policy is X.",
 retrieved_chunks=["The policy is X."],
 ground_truth_answer="The policy is X.",
 ground_truth_chunk_ids=["chunk_1"],
 retrieved_chunk_ids=["chunk_1"],
 client=mock_client,
)
 assert result["score"] is None
 assert result["reasoning"] == "JUDGE_ERROR"

def test_score_clamping():
 mock_client = MagicMock()
 mock_response = MagicMock()
 mock_response.choices = [MagicMock()]
 mock_response.choices[0].message.content = '{"score": 1.5, "reasoning":
"Too high"}'
 mock_client.chat.completions.create.return_value = mock_response

 result = judge_faithfulness(
 question="What is the policy?",
 answer="The policy is X.",
 retrieved_chunks=["The policy is X."],
 ground_truth_answer="The policy is X.",
 ground_truth_chunk_ids=["chunk_1"],
 retrieved_chunk_ids=["chunk_1"],
 client=mock_client,
)
 assert result["score"] == 1.0

```

## I. tests/test\_rag\_bot.py

```

tests/test_rag_bot.py
from unittest.mock import MagicMock, patch

from src.rag_bot import generate_answer, retrieve

def test_generate_answer_uses_context_only():
 mock_client = MagicMock()
 mock_response = MagicMock()
 mock_response.choices = [MagicMock()]
 mock_response.choices[0].message.content = "Test answer"
 mock_client.chat.completions.create.return_value = mock_response

 question = "What is the policy?"
 context_chunks = ["The remote work policy allows 2 days per week."]

 result = generate_answer(question, context_chunks, mock_client)

 call_args = mock_client.chat.completions.create.call_args
 messages = call_args[1]["messages"]
 prompt_content = messages[1]["content"]
 assert "The remote work policy allows 2 days per week." in
prompt_content
 assert "You are a grounded assistant" in messages[0]["content"]
 assert result == "Test answer"

def test_retrieve_returns_correct_structure():
 mock_collection = MagicMock()
 mock_collection.query.return_value = {
 "documents": ["chunk1", "chunk2"],
 "ids": ["id1", "id2"],
 }

 with patch("src.rag_bot._get_embedder") as mock_get_embedder:
 mock_embedder = MagicMock()
 mock_array = MagicMock()
 mock_array.tolist.return_value = [[0.1, 0.2, 0.3]]
 mock_embedder.encode.return_value = mock_array
 mock_get_embedder.return_value = mock_embedder

 result = retrieve("test query", mock_collection, 2)

 assert isinstance(result, tuple)
 assert len(result) == 2
 assert isinstance(result[0], list)
 assert isinstance(result[1], list)
 assert result[0] == ["chunk1", "chunk2"]
 assert result[1] == ["id1", "id2"]

```

## Section 4: Code Logic & Deep-Dive

### 4.1 Judge Prompt Engineering Deep-Dive

**Faithfulness:** The prompt forces the judge to perform a claim-by-claim audit of the generated answer against the retrieved context. The rubric is granular (1.0, 0.7-0.9, 0.4-0.6, 0.1-0.3, 0.0) to prevent the judge from defaulting to binary good/bad judgments. Without this granularity, the judge might conflate minor paraphrasing with outright hallucination, collapsing the signal needed to debug the generator.

**Answer Relevance:** The prompt explicitly provides the ground-truth answer as a reference point, but instructs the judge to evaluate the *generated* answer against the *question*, not against the ground truth. This guards against the failure mode where a factually correct but off-topic answer receives a high score. If the rubric were vague, the judge might score based on fluency rather than question coverage.

**Context Precision:** The prompt surfaces both the retrieved chunk IDs and the ground-truth chunk IDs, giving the judge structural anchors to verify relevance. The rubric distinguishes between "minor noise" (0.7-0.9) and "significant noise" (0.4-0.6), which is critical for diagnosing whether the retriever is returning semantically similar but factually irrelevant chunks—a common failure mode in dense retrieval.

**Context Recall:** The prompt asks the judge to compare ID sets *and* assess content coverage. This dual signal prevents the judge from giving a perfect recall score simply because the retrieved chunks look comprehensive, even if they miss a critical ground-truth chunk. Without explicit ID comparison, the judge might miss structural gaps in retrieval coverage.

### 4.2 JSON Parsing Fragility & Defense

Calling `json.loads()` directly on an LLM response is inherently fragile because production LLMs frequently wrap JSON in markdown fences (`json ...`), add conversational preamble ("Sure, here is the evaluation:"), or append explanatory text after the JSON object. Any of these cause a raw `json.loads()` to raise `JSONDecodeError`.

The defensive pattern used in `_parse_judge_response` implements three layers:

1. **Strip markdown fences:** Detects and removes leading/trailing `orjson` markers.
2. **Direct parse attempt:** Tries `json.loads()` on the cleaned string.
3. **Regex extraction:** If direct parsing fails, uses `re.search(r'\{.*\}', text, re.DOTALL)` to extract the first JSON object, then parses that. This catches cases where the model adds text before or after the JSON.

If all layers fail, the exception propagates to `_call_judge`, which logs the error and returns `{"score": None, "reasoning": "JUDGE_ERROR"}`, ensuring the pipeline continues.

### 4.3 Context Window Budget

The judge model is `llama-3.3-70b-versatile` with a 128,000-token context window. We must ensure that a single judge call never exceeds this limit.

#### Overhead calculation:

- System prompt + judge template instructions: ~800 tokens
- Question text: ~100 tokens

- Generated answer: ~500 tokens
- Ground-truth answer: ~500 tokens
- Chunk ID lists (retrieved + ground truth): ~200 tokens
- Safety margin for JSON response and prompt overhead: ~1,000 tokens
- **Total overhead:** ~3,100 tokens

**Available for retrieved chunks:**  $128,000 - 3,100 = 124,900$  tokens

**Per-chunk cost:** `CHUNK_SIZE = 512` words. Using the `count_tokens` heuristic (`words * 1.3`):  $512 * 1.3 = \sim 665$  tokens per chunk

**Maximum safe chunks:**  $124,900 / 665 \approx 187$  chunks

With `TOP_K_RETRIEVAL = 5`, the judge prompt consumes only ~3,325 tokens for chunks, leaving ~120,000 tokens unused. This means the pipeline could safely increase `top_k` to 50+ without approaching the context limit, though latency and cost would rise.

## 4.4 ChromaDB Query Flow

The retrieval pipeline executes the following steps when `retrieve()` is called:

1. **Query Embedding:** The raw query string is passed to `SentenceTransformer.encode()`, which tokenizes the text, runs it through the `all-MiniLM-L6-v2` transformer, and outputs a 384-dimensional dense vector.
2. **Collection Query:** The query vector is passed to `collection.query(query_embeddings=[...], n_results=top_k)`. ChromaDB computes the cosine similarity between the query vector and every stored chunk embedding in the `rag_corpus` collection.
3. **Similarity Ranking:** ChromaDB ranks all chunks by cosine similarity (descending) and returns the `top_k` results.
4. **Result Extraction:** The `documents` and `ids` arrays are extracted from the query result. These represent the text content and deterministic IDs of the most semantically similar chunks.

The role of `top_k` is to bound the search results to the most relevant chunks. A higher `top_k` increases recall (more chance of surfacing the right chunk) but decreases precision (more noise for the generator) and increases token consumption in the generator and judge prompts.

## 4.5 avg\_score Calculation

The `avg_score` is computed as a simple arithmetic mean of the four metrics, ignoring any `None` values (failed judge calls). This is convenient because it provides a single scalar for ranking and requires no domain-specific tuning.

However, a simple mean is potentially misleading because it treats all four metrics as equally important. In a production system, **Faithfulness** and **Answer Relevance** are often more critical than retrieval metrics: a faithful but irrelevant answer is useless, and a relevant but hallucinated answer is dangerous. A weighted average might look like:

```
weighted_score = (0.35 * faithfulness) + (0.35 * answer_relevance) + (0.15
* context_precision) + (0.15 * context_recall)
```

This weighting reflects the principle that the end-user experience is dominated by the quality of the final answer, while retrieval is an upstream enabler. If the system optimizes for `avg_score` without weighting, it might over-invest in retrieval tuning while neglecting generator grounding.

## Section 5: Deployment & Execution Guide

### 5.1 Environment Setup

```
python3 -m venv myenv
source myenv/bin/activate # Linux/Mac
myenv\Scripts\activate # Windows
pip install --upgrade pip
pip install -r requirements.txt
```

### 5.2 Environment Variable Configuration

Create a `.env` file in the project root:

```
.env
GROQ_API_KEY=gsk_XX
```

`python-dotenv` loads this automatically. In `eval/run_eval.py`, the call is placed at the top of `main()`:

```
from dotenv import load_dotenv
load_dotenv()
```

**NEVER commit `.env` to git. Verify `.gitignore` contains `.env`.** Run:

```
git check-ignore -v .env
```

If this returns nothing, your `.env` is NOT ignored and you must fix `.gitignore` immediately.

### 5.3 Step-by-Step Execution Order

```
Step 1: Drop corpus .txt files into data/raw/
Step 2: python src/ingest.py # Build the ChromaDB index
Step 3: Populate eval/test_set.jsonl # Complete all 30 questions manually
```

```
Step 4: python eval/run_eval.py # Run the full evaluation
Step 5: Check eval/results.csv # Verify output
Step 6: Open eval/eval_report.md # Read the generated report
```

## 5.4 Verification Checklist

- ☐ `output.log` exists and contains timestamped entries
- ☐ `eval/results.csv` has exactly 30 rows + 1 header row
- ☐ All 4 score columns contain values between 0.0 and 1.0 (or `null` for failures)
- ☐ `eval/eval_report.md` contains the histogram image reference
- ☐ `eval/score_distribution.png` exists
- ☐ `tests/` pass with `pytest tests/ -v`
- ☐ No `.env` file in git history (`git log --all -- .env` returns empty)

## 5.5 Running Tests

```
pytest tests/ -v --tb=short
```

---

## Section 6: Intern Viva & Code Review Questions

### ## Project Evaluation & Code Review

### Q1: Explain the difference between Faithfulness and Answer Relevance. Why are both necessary, and what failure mode would occur if we only measured one?

**\*\*Answer:\*\***

Faithfulness measures whether the generated answer is factually grounded in the retrieved context (no hallucination). Answer Relevance measures whether the answer actually addresses the question asked. Both are necessary because an answer can be perfectly faithful to irrelevant context (high faithfulness, low relevance) or can be a fluent, relevant-sounding hallucination (high relevance, low faithfulness). Measuring only one would mask critical failure modes: a system could score well on faithfulness by generating safe, context-bound trivia that never answers the user's actual question, or score well on relevance by confidently hallucinating a plausible answer.

### Q2: Why do we use an LLM-as-judge instead of rule-based metrics like BLEU or ROUGE for this RAG evaluation harness?

**\*\*Answer:\*\***

Rule-based metrics like BLEU and ROUGE measure lexical n-gram overlap between the generated answer and a reference. They are brittle to paraphrasing, synonym substitution, and valid answers that use different wording from the ground truth. LLM-as-judge evaluates semantic meaning, factual correctness, and coverage—capabilities essential for RAG where answers are grounded in retrieved context rather than memorized training data. An LLM judge can verify that a claim is supported by the context even

if the wording differs completely from the reference answer.

### Q3: What defensive measures does `\_parse\_judge\_response` implement, and why is each layer necessary?

**\*\*Answer:\*\***

`\_parse\_judge\_response` implements three defensive layers: (1) stripping markdown code fences (``json or ``) because LLMs frequently wrap JSON in markdown formatting; (2) a direct `json.loads()` attempt on the cleaned string for the ideal case; (3) a regex fallback `re.search(r'\{.\*\}', text, re.DOTALL)` to extract the first JSON object when the model adds conversational preamble or trailing text. Each layer is necessary because production LLM outputs are non-deterministic in formatting. Without all three, a single markdown fence or "Here is the evaluation:" preamble would crash the parser and abort the metric for that question.

### Q4: Walk through the exact steps from a query string entering `retrieve()` to ChromaDB returning the top-k chunks. What mathematical operation happens at each step?

**\*\*Answer:\*\***

Step 1: The query string is tokenized and embedded by `SentenceTransformer.encode()`, producing a 384-dimensional dense vector via the `all-MiniLM-L6-v2` model. Step 2: This vector is passed to `collection.query()` as the `query\_embeddings`. Step 3: ChromaDB computes the cosine similarity between the query vector and every stored chunk vector in the `rag\_corpus` collection. Cosine similarity is defined as the dot product of the two normalized vectors. Step 4: ChromaDB ranks all chunks by similarity score in descending order. Step 5: The top `n\_results=top\_k` chunks are returned, along with their IDs and document text.

### Q5: Explain the context window budget calculation for this project. If we increased CHUNK\_SIZE to 2048 words, how would that affect the maximum safe top\_k?

**\*\*Answer:\*\***

The judge model (`llama-3.3-70b-versatile`) has a 128k-token context. Overhead (prompt template, question, answer, ground truth, IDs, safety margin) consumes ~3,100 tokens, leaving ~124,900 tokens for chunks. At the current CHUNK\_SIZE of 512 words, each chunk costs ~665 tokens (512 \* 1.3), yielding a theoretical maximum of ~187 chunks. If CHUNK\_SIZE increased to 2048 words, each chunk would cost ~2,662 tokens. The maximum safe top\_k would drop to 124,900 / 2,662 ≈ 46 chunks. While still large, this reduces headroom and increases per-call latency and cost.

### Q6: Describe the graceful degradation strategy in this pipeline. If the Groq API returns a 503 error during the 17th question's faithfulness judge, what exact state is recorded, and how does the pipeline continue?

**\*\*Answer:\*\***

The pipeline implements lenient graceful degradation at two levels. First, every Groq API call is wrapped in a `try/except` block. If the faithfulness judge for question 17 fails, `judge\_faithfulness` logs the exception at ERROR level and returns `{"score": None, "reasoning": "JUDGE\_ERROR"}`. Second, `run\_single\_evaluation` aggregates this result into the result dict, and `main()` counts this as a partial failure rather than a fatal error. The pipeline continues to question 18 immediately. At the end of the

run, the summary log reports `16/30 questions fully scored, 14 had partial failures` (or similar). The CSV will contain `null` for the faithfulness score of question 17, but all other metrics and questions are preserved.

### Q7: Analyze a failure mode where the LLM judge consistently scores faithfulness 0.1–0.2 points higher than a human expert would. What could cause this, and how would you detect it?

**\*\*Answer:\*\***

This systematic inflation could be caused by (1) **\*\*leniency bias\*\*** in the judge prompt, where the rubric language ("minor unsupported inferences") is interpreted generously; (2) **\*\*inbreeding bias\*\***, where the judge model (same LLaMA family as the generator) recognizes its own stylistic patterns and assumes they are correct; or (3) **\*\*context anchoring\*\***, where the judge overweights the presence of key phrases in the context even if the answer misapplies them. Detection requires a **\*\*human calibration audit\*\***: sample 10–20 evaluations across the score range, have a human expert independently score them, and compute the Pearson correlation and mean absolute error between human and judge scores. If the correlation is high but the judge mean is consistently offset, the prompt rubric needs tightening.

### Q8: The eval harness judges the RAG system, but who evaluates the evaluator? Propose a meta-evaluation strategy to validate that the 4 judge prompts are actually measuring what they claim to measure.

**\*\*Answer:\*\***

Meta-evaluation requires constructing a **\*\*synthetic benchmark set\*\*** with known, controlled defects. For each metric, create a grid of test cases:

- Faithfulness: answers that are (a) fully supported, (b) partially hallucinated, (c) entirely contradicted.
- Answer Relevance: answers that are (a) fully responsive, (b) partially off-topic, (c) completely unrelated.
- Context Precision: retrieved sets that are (a) all relevant, (b) 50% noise, (c) all irrelevant.
- Context Recall: retrieved sets that are (a) complete, (b) missing one critical chunk, (c) missing all chunks.

Run the judges against this labeled set and compute accuracy, precision, and recall against the known labels. If the judge fails to distinguish these controlled classes, the prompt rubric or model temperature needs adjustment. This is analogous to evaluating a classifier against a gold-standard test set before deploying it.

### Q9: This system uses the same LLM family (llama-3.3-70b) for both generation and judging. Explain the systemic bias risks of self-evaluation (inbreeding bias) and propose 2 concrete mitigation strategies that do not involve simply switching to a different model provider.

**\*\*Answer:\*\***

**\*\*Inbreeding bias\*\*** occurs when the judge model shares the generator's training distribution, tokenization biases, and stylistic preferences. It may fail to catch hallucinations that align with its own generation patterns, or it may penalize answers that deviate from its preferred phrasing even when correct. Two concrete mitigations: (1) **\*\*Adversarial prompt hardening\*\***: In the judge prompt, explicitly instruct the model to "assume the generated answer was written by a different AI system and verify every claim as if it were suspect." This cognitive framing reduces

self-affinity. (2) **Ensemble judging with temperature variation**: Run the same judge prompt 3 times with temperature=0.7 and aggregate via median score. High variance across runs indicates judge uncertainty, flagging the result for human review. This does not require a different provider but adds statistical rigor.

*### Q10: Goodhart's Law states that when a measure becomes a target, it ceases to be a good measure. If the intern optimizes the RAG system exclusively to maximize `avg\_score`, describe three ways the system could game the metric while degrading real-world utility. Then propose a statistically rigorous replacement for `avg\_score` that is harder to game.*

**Answer:**

Three gaming strategies: (1) **Prompt stuffing**: The generator could be instructed to repeat large verbatim passages from the retrieved context, artificially inflating faithfulness and context precision while producing unreadable, low-utility answers. (2) **Question overfitting**: The retrieval system could be tuned to surface chunks that contain keywords from the known test questions, memorizing the eval set rather than generalizing to novel queries. (3) **Answer length manipulation**: The generator could produce extremely short answers ("Yes" or "No") that are easy to judge as relevant and faithful but provide no actionable detail to the user.

A statistically rigorous replacement is a **utility-weighted composite with human-calibrated thresholds**. Replace `avg\_score` with:

$$U = w_f * \min(\text{faithfulness}, 0.95) + w_r * \min(\text{answer\_relevance}, 0.95) + w_p * \text{context\_precision} + w_c * \text{context\_recall} - \text{penalty}(\text{length} < 20 \text{ words}) - \text{penalty}(\text{faithfulness} > 0.99 \text{ AND } \text{answer\_relevance} < 0.5)$$

The `min(..., 0.95)` caps prevent perfect scores from dominating. The penalties explicitly punish the known gaming modes (overly short answers and faithful-but-irrelevant outputs). The weights `w\_f, w\_r, w\_p, w\_c` are derived from a logistic regression on human-labeled "useful / not useful" judgments, making the metric a proxy for human utility rather than an arbitrary average. This transforms the target from a simple average into a calibrated predictor of human satisfaction.

<!-- `` -->